doc.doc.md

Transform your document collections into organized, searchable markdown files.

doc.doc.md is a command-line tool that processes document collections in directory structures and generates markdown files for each document. Perfect for managing personal documents, home labs, or any collection where you want markdown-based organization without a complex document management system.

⚠️ Development Status

This project is in early development. The core structure is in place, but the main `doc.doc.sh` script is not yet fully implemented. Contributions and feedback are welcome!

Features

- **Directory Mirroring**: Automatically mirrors your input directory structure in the output location
- **Markdown Generation**: Creates markdown files for each document based on customizable templates
- **Advanced Filtering**: Powerful include/exclude logic with AND/OR operators for extensions, glob patterns, and MIME types
- **Language-Agnostic Plugin System**: Extensible architecture supporting plugins in any language (Bash, Python, compiled binaries)
- **Template-Based Output**: Full control over the structure of generated markdown files
- **Unix Pipeline Architecture**: Efficient file processing using standard Unix pipes and streams
- **Simple CLI**: Straightforward command-line interface with clear options

Installation

Prerequisites

- **Bash** shell environment (version 4.0+)
- **jq** (JSON processor, used by built-in plugins)
- **Python 3.12+** for advanced filtering and processing

- **Git** (for cloning the repository)

The tool uses a mixed Bash+Python architecture:

- Bash handles CLI orchestration, file discovery, and plugin execution
- Python handles complex filtering logic (AND/OR operators across include/exclude parameters)

Quick Start

1. Clone the repository:

```
git clone https://github.com/PeculiarMind/doc.doc.md.git
cd doc.doc.md
```

2. Make the script executable:

```
chmod +x doc.doc.sh
```

3. Run your first command:

```
./doc.doc.sh --help
```

Development Container

This project is configured for use with VS Code Dev Containers. If you're using VS Code with the Remote-Containers extension, simply open the repository and select "Reopen in Container" when prompted.

Usage

Basic Command

```
./doc.doc.sh process \
  --input-directory /path/to/documents \
  --output-directory /path/to/output \
  --template /path/to/template.md
```

Advanced Filtering

Filter files using powerful include/exclude logic:

```
./doc.doc.sh process \
  --input-directory /path/to/documents \
  --output-directory /path/to/output \
```

```
  --include ".pdf,.docx" \
  --include "**/2024/**" \
  --exclude ".log" \
  --exclude "**/temp/**" \
  --exclude "**/drafts/**"
```

Filter Logic:

- Multiple `--include` parameters are ANDed (file must match at least one criterion from each)
- Values within a single `--include` parameter are ORed (e.g., `.pdf,.docx` matches either)
- Same logic applies to `--exclude` parameters
- Auto-detects filter types: file extensions (`.pdf`), glob patterns (`**/2024/**`), or MIME types (`application/pdf`)

MIME Type Filtering

Criteria containing `/` (but not `**`) are treated as MIME type criteria, evaluated against the MIME type detected by the `file` plugin:

```
# Include only plain-text files
./doc.doc.sh process \
  --input-directory /path/to/documents \
  --output-directory /path/to/output \
  --include "text/plain"

# Exclude all image files using a wildcard MIME pattern
./doc.doc.sh process \
  --input-directory /path/to/documents \
  --output-directory /path/to/output \
  --exclude "image/*"

# Combine MIME type and extension filters
./doc.doc.sh process \
  --input-directory /path/to/documents \
  --output-directory /path/to/output \
  --include "application/pdf,.docx" \
  --exclude "image/*"
```

MIME filter behaviour:

- Wildcard patterns are supported: `image/*` matches `image/png` , `image/jpeg` , etc.
- Files that do not pass the MIME filter are **silently skipped** (no output, no error)

- MIME detection requires the `file` plugin to be installed and active; if it is not, an error is raised and processing stops

> **Note:** The `file` plugin always runs first in the processing chain to ensure MIME type information is available for all subsequent filters and plugins.

Example Workflow

Input Directory:

```
/documents
├── 2025
│   ├── somedoc.pdf
│   ├── anotherdoc.docx
│   └── justaphoto.jpg
└── 2024
    └── olderdoc.pdf
```

Output Directory (after processing):

```
/output
├── 2025
│   ├── somedoc.md
│   ├── anotherdoc.md
│   └── justaphoto.md
└── 2024
    └── olderdoc.md
```

Each `.md` file contains metadata and content extracted from the original document, formatted according to your template.

Command-Line Options

Process Command

Option	Short	Description	Required	Default
`--input-directory`	`-d`	Path to the input directory containing documents	Yes	
`--output-directory`	`-o`	Path where the markdown files will be created	Yes	
`--template`	`-t`	Path to the markdown template file	No	Built-in default

Option	Short	Description	Required	Default
`--include`	`-i`	Comma-separated file extensions, glob patterns, or MIME types to include	No	All files
`--exclude`	`-e`	Comma-separated file extensions, glob patterns, or MIME types to exclude	No	

Plugin Commands

```
./doc.doc.sh list plugins                    # List all available plugins
./doc.doc.sh list plugins active             # List active plugins only
./doc.doc.sh list plugins inactive           # List inactive plugins only
./doc.doc.sh activate --plugin <name>        # Activate a plugin
./doc.doc.sh deactivate --plugin <name>      # Deactivate a plugin
./doc.doc.sh install --plugin <name>         # Install plugin dependencies
./doc.doc.sh installed --plugin <name>       # Check if plugin is installed
./doc.doc.sh tree                            # Display plugin dependency tree
```

Project Structure

```
doc.doc.md/
├── doc.doc.sh              # Main script
├── doc.doc.md/             # Core directory
│   ├── components/         # Reusable components (planned)
│   ├── plugins/            # Plugin directory
│   │   ├── stat/           # File statistics plugin
│   │   │   ├── descriptor.json
│   │   │   ├── main.sh
│   │   │   ├── install.sh
│   │   │   └── installed.sh
│   │   └── file/           # MIME type detection plugin
│   │       ├── descriptor.json
│   │       ├── main.sh
│   │       ├── install.sh
│   │       └── installed.sh
│   └── templates/          # Template directory
│       └── default.md      # Default markdown template
├── project_documentation/  # Project docs (arc42 structure)
├── project_management/     # Guidelines, workflows, tools
└── LICENSE.md             # AGPL-3.0 license
```

Plugins

Plugins extend doc.doc.md's functionality by extracting metadata and content from different file types. The project includes built-in plugins for file statistics and MIME type detection.

Built-in Plugins

- **file**: Detects MIME types using the standard `file` command — **always runs first** in the processing chain; must be installed and active
- **stat**: Extracts file system metadata (size, owner, timestamps)

Plugin Architecture

Plugins are **language-agnostic** and invoked via shell commands:

- Defined using `descriptor.json` files
- Can be implemented in **any language** (Bash, Python, compiled binaries, etc.)
- Invoked as shell commands, never imported directly
- Simple interface: receive JSON input via stdin, produce JSON output via stdout
- Type-safe communication using JSON for both input and output

Plugin Structure

Plugins are defined using JSON descriptors that specify:

- **Name and description**: Plugin identification
- **Commands**: Shell commands to execute (main, install, installed checks)
- **Input parameters**: What the plugin needs to process a file
- **Output fields**: What information the plugin extracts

Example: file Plugin (Bash)

The `file` plugin detects MIME types using the standard `file` command:

```
{
  "name": "file",
  "description": "Detects MIME types using the file command",
  "commands": {
    "process": {
      "command": "main.sh",
      "input": {
        "filePath": { "required": true, "type": "string", "description": "Path
to file" }
      },
      "output": {
        "mimeType": { "type": "string", "description": "Detected MIME type" }
```

```
      }
    },
    "install": {
      "command": "install.sh"
    },
    "installed": {
      "command": "installed.sh"
    }
  }
}
```

The plugin receives input via stdin:

```
{"filePath": "/path/to/file.pdf"}
```

Example: stat Plugin (Bash)

The `stat` plugin extracts file system information:

```
{
  "name": "stat",
  "description": "Provides statistical information about a file",
  "commands": {
    "process": {
      "command": "main.sh",
      "input": {
        "filePath": { "required": true, "type": "string", "description": "Path
to file" }
      },
      "output": {
        "fileSize": { "type": "number", "description": "File size in bytes" },
        "fileOwner": { "type": "string", "description": "File owner" },
        "fileCreated": { "type": "string", "description": "File creation date
(ISO 8601)" },
        "fileModified": { "type": "string", "description": "Last modification
date (ISO 8601)" },
        "fileMetadataChanged": { "type": "string", "description": "Metadata
change date (ISO 8601)" }
      }
    },
    "install": {
      "command": "install.sh"
    },
    "installed": {
      "command": "installed.sh"
```

```
    }
  }
}
```

The plugin receives input via stdin:

```
{"filePath": "/path/to/file.pdf"}
```

Example: Python Plugin

Plugins can be implemented in Python:

```
{
  "name": "pdf-extractor",
  "commands": {
    "process": {
      "command": "python3 extract.py",
      "input": {
        "filePath": { "required": true, "type": "string", "description": "Path
to PDF" }
      },
      "output": {
        "pdfTitle": { "type": "string", "description": "PDF title" },
        "pdfAuthor": { "type": "string", "description": "PDF author" }
      }
    }
  }
}
```

The Python script reads JSON from stdin:

```
import json
import sys

input_data = json.load(sys.stdin)
file_path = input_data['filePath']
# Process file...
output = {"pdfTitle": "...", "pdfAuthor": "..."}
json.dump(output, sys.stdout)
```

Creating Custom Plugins

1. Create a new directory under `doc.doc.md/plugins/[plugin-name]/`

2. Add a `descriptor.json` file defining commands, inputs, and outputs
3. Implement the plugin in your preferred language:
   - **Bash**: Create `main.sh` script
   - **Python**: Create `process.py` script
   - **Compiled**: Add your binary and update command path
4. Specify the shell command in `descriptor.json`
5. Optionally add `install.sh` and `installed.sh` for dependency management
6. Reference the plugin output variables in your template

Templates

Templates control the structure and content of generated markdown files.

Default Template

Located at `doc.doc.md/templates/default.md`:

```
# {{fileName}}

Document processed on {{processingDate}}
```

Template Variables

Templates support variables populated by plugins. Variables use the `{{variableName}}` syntax (lowerCamelCase).

Example template using the stat plugin:

```
# {{fileName}}

## File Information

- **Size**: {{fileSize}} bytes
- **Owner**: {{fileOwner}}
- **Created**: {{fileCreated}}
- **Modified**: {{fileModified}}
```

Creating Custom Templates

1. Create a new `.md` file with your desired structure

2. Use `{{variableName}}` placeholders for dynamic content (lowerCamelCase matching plugin outputs)
3. Pass the template path via the `--template` option

Use Cases

- **Personal Document Management**: Convert your file cabinets into searchable markdown
- **Home Lab Documentation**: Generate documentation indices for your digital archives
- **Content Migration**: Prepare documents for import into Obsidian, Logseq, or other markdown tools
- **Document Cataloging**: Create metadata-rich catalogs of large document collections

Architecture

doc.doc.md uses a **mixed Bash and Python architecture** for optimal performance and maintainability:

- **Bash**: CLI orchestration, file discovery (`find`), plugin execution, Unix pipeline coordination
- **Python 3.12+**: Complex filtering logic with AND/OR operators across include/exclude parameters

Why this approach?

- **Primary driver**: Complex filter evaluation (AND/OR logic across multiple parameters) is difficult to implement reliably in pure Bash
- Leverages each language's strengths (shell operations vs. complex logic)
- Maintains Unix CLI tool philosophy
- Supports language-agnostic plugins via shell command invocation
- Efficient pipeline processing via null-delimited streams

See ADR-001 for detailed rationale.

Contributing

Contributions are welcome! This project is in active development and needs help with:

- Core script implementation (Bash + Python components)
- Plugin development (PDF, DOCX, image processing, etc.) in any language
- Template creation
- Documentation improvements
- Testing and bug reports

Getting Started

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

Development Guidelines

- Follow the workflow guidelines in `project_management/01_guidelines/workflows/`
- Review the communication standards in `project_management/01_guidelines/agent_behavior/`
- Check the documentation standards in `project_management/01_guidelines/documentation_standards/`

License

This project is licensed under the **GNU Affero General Public License v3.0 (AGPL-3.0)**.

Key points:

- **Open Source**: Source code must be made available
- **Copyleft**: Derivative works must use the same license
- **Network Use**: Running modified versions on a server requires sharing source code with users
- **Patent Grant**: Contributors grant patent rights to users

See LICENSE.md for full details.

Credits

- **arc42**: Architecture documentation structure based on the arc42 template
- **ProTemp.AI**: Project structure inspired by PeculiarMind/ProTemp.AI

See CREDITS.md for complete attribution.

Support

- **Issues**: Report bugs or request features via GitHub Issues
- **Discussions**: Join conversations about the project
- **Documentation**: Comprehensive guides in `project_documentation/`

Ready to organize your documents? Start by exploring the examples and customizing your first template!